Studio1 Report

written by Zihan Chen

test1

studio1.cpp

```
#include "struct.h"
#include <iostream>

using namespace std;

const int SUCCESS = 0;

int main() {
    Studio1_Struct x = Studio1_Struct{1};
    Studio1_Struct y = Studio1_Struct{2};

    cout << x.value << " " << y.value << endl;

    return SUCCESS;
}
```

struct.h

```
#ifndef STRUCT_H
#define STRUCT_H

struct Studio1_Struct{
    int value;
    Studio1_Struct(int v);
};

#endif
```

struct.cpp

```
#include "struct.h"

Studio1_Struct::Studio1_Struct(int v) : value(v){}
```

Output:

```
1 2
```

test2

As the book mentioned.

> It is not possible to define an object or delete a pointer to a dynamically allocated object of a type with a deleted destructor.

```
    // suppress the compiler's synthesis
    Studio1_Struct(const Studio1_Struct&) = delete; // (1) copy
constructor
    Studio1_Struct& operator=(const Studio1_Struct&) = delete; // (2) copy
assignment constructor

    // Destructor is a must, if we don't have destructor, then we can't
delete the Studio1_Struct.
    //~Studio1_Struct() = delete; // (3) destruct
```

test3

Add the code in studio1.cpp.

```
    swap(x, y);

    cout << x.value << " "  << y.value << endl;
```

Commnet out the previous delete, or it won't run successfully.

```
    //Studio1_Struct(const Studio1_Struct&) = delete; // (1) copy
constructor
    //Studio1_Struct& operator=(const Studio1_Struct&) = delete; // (2)
copy assignment constructor
```

Result

```
1 2
2 1
```

test4

```
    // suppress the compiler's synthesis
    // Copy constructor and copy assignment constructor will calling by
swap while running, so they can't be deleted.
    //Studio1_Struct(const Studio1_Struct&) = delete; // (1) copy
constructor
    //Studio1_Struct& operator=(const Studio1_Struct&) = delete; // (2)
copy assignment constructor

    // Destructor is a must, if we don't have destructor, then we can't
delete the Studio1_Struct.
    //~Studio1_Struct() = delete; // (3) destruct
```

test5

struct.cpp

```
#ifndef STRUCT_CPP
#define STRUCT_CPP

#include "struct.h"

template <typename T>
Studio1_Struct<T>::Studio1_Struct(T v) : value(v){}
#endif
```

struct.h

```
#ifndef STRUCT_H
#define STRUCT_H

template <typename T = int>
struct Studio1_Struct{
    public: T value;
    public: Studio1_Struct(T v);

    // suppress the compiler's synthesis
    // Copy constructor and copy assignment constructor will calling by
swap while running, so they can't be deleted.
    //Studio1_Struct(const Studio1_Struct&) = delete; // (1) copy
constructor
    //Studio1_Struct& operator=(const Studio1_Struct&) = delete; // (2)
copy assignment constructor

    // Destructor is a must, if we don't have destructor, then we can't
delete the Studio1_Struct.
    //~Studio1_Struct() = delete; // (3) destruct
};
```

```
#include "struct.cpp"
#endif
```

In studio1.cpp, make some change

```
    Studio1_Struct<> x(1);
    Studio1_Struct<> y(2);
```

Result

```
1 2
2 1
```

test6

struct.h

```
#ifndef STRUCT_H
#define STRUCT_H
#include <iostream>
using std::ostream;

template <typename T = int>
class Studio1_Struct{
    private: T value;
    public:
        Studio1_Struct(T v);

        template <typename U>
        friend ostream &operator<<(ostream &os, const Studio1_Struct<U>
&item);

    // suppress the compiler's synthesis
    // Copy constructor and copy assignment constructor will calling by
swap while running, so they can't be deleted.
    //Studio1_Struct(const Studio1_Struct&) = delete; // (1) copy
constructor
    //Studio1_Struct& operator=(const Studio1_Struct&) = delete; // (2)
copy assignment constructor

    // Destructor is a must, if we don't have destructor, then we can't
delete the Studio1_Struct.
    //~Studio1_Struct() = delete; // (3) destruct
};

#include "struct.cpp"
#endif
```

struct.cpp

```
#ifndef STRUCT_CPP
#define STRUCT_CPP

#include "struct.h"

template <typename T>
Studio1_Struct<T>::Studio1_Struct(T v) : value(v){}

template <typename T>
ostream &operator<<(ostream &os, const Studio1_Struct<T> &item) {
    os << item.value;
    return os;
}

#endif
```

studio1.cpp

```
#include "struct.h"
#include <iostream>

using namespace std;

const int SUCCESS = 0;

int main() {
    Studio1_Struct<> x(1);
    Studio1_Struct<> y(2);

    cout << x << " " << y << endl;

    swap(x, y);

    cout << x<< " "  << y << endl;

    return SUCCESS;
}
```

Result

```
1 2
2 1
```